

A - Used Car Selection

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 233 pts

Problem Statement

Takahashi is visiting a used car dealership to purchase his first car. The dealership has N used cars, and for each car, the "price" and "mileage" are known. The i -th car has a price of P_i ten-thousand yen and a mileage of K_i km.

Due to budget constraints, Takahashi cannot purchase a car that is too expensive. On the other hand, he also wants to avoid cars that are too cheap, as he is concerned about their quality. Furthermore, he wants to avoid cars with too high a mileage due to the increased risk of breakdowns.

Specifically, Takahashi defines a car as a "candidate car" if it satisfies all of the following conditions:

- The price is at least L ten-thousand yen and at most R ten-thousand yen (i.e., $L \leq P_i \leq R$)
- The mileage is at most M km (i.e., $K_i \leq M$)

Among the N used cars, find the number of cars that qualify as "candidate cars." Each car is counted individually.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq L \leq R \leq 10^4$
- $1 \leq M \leq 10^6$
- $1 \leq P_i \leq 10^4$ ($1 \leq i \leq N$)
- $1 \leq K_i \leq 10^6$ ($1 \leq i \leq N$)
- All inputs are integers

Input

```
 $N$   $L$   $R$   $M$   
 $P_1$   $K_1$   
 $P_2$   $K_2$   
 $\vdots$   
 $P_N$   $K_N$ 
```

- The first line contains the number of used cars N , the lower bound of price L , the upper bound of price R , and the upper bound of mileage M , separated by spaces.
- Each of the following N lines, where the i -th line ($1 \leq i \leq N$), contains the price P_i and mileage K_i of the i -th car, separated by spaces.

Output

Output the number of "candidate cars" in a single line.

Sample Input 1

[Copy](#)

```
5 100 200 50000
150 30000
250 20000
80 10000
200 50000
180 60000
```

Sample Output 1

[Copy](#)

```
2
```

Sample Input 2

[Copy](#)

```
4 150 300 40000
100 30000
500 20000
200 80000
50 50000
```

Sample Output 2

[Copy](#)

```
0
```

Sample Input 3

[Copy](#)

```
10 50 150 100000
120 80000
50 100000
200 50000
30 90000
150 100001
75 60000
100 120000
90 99999
160 30000
45 100000
```

Sample Output 3

[Copy](#)

```
4
```

Sample Input 4

[Copy](#)

```
20 200 500 80000
300 50000
150 40000
450 80000
500 80001
200 79999
600 30000
250 90000
350 70000
199 60000
501 50000
400 80000
210 1
180 10000
320 75000
275 100000
480 60000
200 80000
100 20000
500 50000
300 80000
```

Sample Output 4

[Copy](#)

```
11
```

Sample Input 5

[Copy](#)

```
1 1 1 1
1 1
```

Sample Output 5

[Copy](#)

```
1
```

B - Assortment of Sweets

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 333 pts

Problem Statement

Takahashi works at a candy shop. The shop has N candies lined up in a row from left to right, and the i -th candy from the left ($1 \leq i \leq N$) has a "deliciousness" value of A_i .

Takahashi will now make exactly M assortment sets. Each set is constructed by choosing one contiguous interval from the row of candies. Specifically, he chooses a pair of integers (l, r) ($1 \leq l \leq r \leq N$) and bundles together the candies numbered $l, l+1, \dots, r$. However, the length $r-l+1$ of each interval must be at most K .

When making the M sets, all M chosen intervals must be distinct. That is, for any two sets, their corresponding pairs (l, r) must not be identical. On the other hand, intervals of different sets are allowed to overlap, meaning the same candy may be included in multiple sets.

The "satisfaction" of each set is defined as the sum of deliciousness values of the candies included in that set. That is, the satisfaction of a set with interval (l, r) is $A_l + A_{l+1} + \dots + A_r$.

Among all ways to make exactly M sets, find and output the maximum possible total satisfaction of the M sets.

It is guaranteed that the total number of intervals (l, r) with length at most K is at least M , so it is always possible to choose M distinct intervals satisfying the conditions.

Constraints

- $1 \leq N \leq 3000$
- $1 \leq K \leq N$
- $1 \leq M \leq \sum_{k=1}^K (N - k + 1)$ (the right-hand side equals the total number of intervals with length at most K)
- $-10^6 \leq A_i \leq 10^6$ ($1 \leq i \leq N$)
- All input values are integers.

Input

```
 $N$   $M$   $K$   
 $A_1$   $A_2$   $\dots$   $A_N$ 
```

- The first line contains the integer N representing the number of candies, the integer M representing the number of sets to make, and the integer K representing the upper limit on the number of candies that can be included in one set, separated by spaces.
- The second line contains the integers $A_1, A_2, \dots, A_N$ representing the deliciousness of each candy, separated by spaces.

Output

Output the maximum possible total satisfaction of the M sets in a single line.

Sample Input 1

[Copy](#)

```
5 3 2  
3 -1 4 1 5
```

Sample Output 1

[Copy](#)

```
16
```

Sample Input 2

[Copy](#)

```
7 5 3  
2 -3 5 -1 8 -2 4
```

Sample Output 2

[Copy](#)

```
43
```

Sample Input 3

[Copy](#)

```
10 4 4  
1 -2 3 4 -5 6 7 -8 9 1
```

Sample Output 3

[Copy](#)

```
49
```

C - Minimum Number of Items to Buy

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 366 pts

Problem Statement

Takahashi is shopping at a supermarket. There are N items lined up in a row on the shelf, and the price of the i -th item from the left is A_i yen. There is only one of each item.

Takahashi wants to select some of these items and put them in his basket. For each item, he decides whether to "select" it or "not select" it, and he wants the total price of the selected items to be exactly S yen.

However, Takahashi wants to achieve a total of exactly S yen with as few items as possible.

Determine whether it is possible to make the total price of the selected items exactly S yen. If it is possible, output the minimum number of items to select. If it is impossible, output -1 .

Constraints

- $1 \leq N \leq 100$
- $1 \leq S \leq 10000$
- $1 \leq A_i \leq 10000$ ($1 \leq i \leq N$)
- All inputs are integers

Input

```
N S
A_1 A_2 ... A_N
```

- The first line contains N , representing the number of items, and S , representing the target total price, separated by a space.
- The second line contains the prices of each item $A_1, A_2, \dots, A_N$, separated by spaces.

Output

If it is possible to make the total price of the selected items exactly S yen, output the minimum number of items to select on a single line. If it is impossible, output -1 .

Sample Input 1

[Copy](#)

```
5 10
3 5 7 2 8
```

Sample Output 1

[Copy](#)

```
2
```

Sample Input 2

[Copy](#)

```
4 11
3 6 9 12
```

Sample Output 2

[Copy](#)

```
-1
```

Sample Input 3

[Copy](#)

```
15 100
12 50 25 7 33 18 50 44 9 61 15 73 28 36 42
```

Sample Output 3

[Copy](#)

```
2
```


D - Fluffy Cloud Hopping

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 400 pts

Problem Statement

There are N fluffy clouds floating in the sky, numbered from 1 to N . There are M wind paths between the clouds, each of which can be traversed in both directions.

Each cloud i has a fluffiness level F_i (an integer between 1 and K , inclusive). The fluffiness level of each cloud is a fixed value and does not change throughout the game.

Wind path j connects cloud U_j and cloud V_j , and has a base travel time T_j .

Takahashi starts at cloud 1 and aims to reach cloud N by following wind paths. He may pass through the same cloud or the same wind path multiple times.

Takahashi has a state called "current fluffiness level," whose initial value is F_1 , the fluffiness level of cloud 1. When Takahashi traverses wind path j with a current fluffiness level of f , the time it takes is $T_j \times f$. The fluffier he is, the more wind resistance he encounters, and the longer the travel takes.

Each time Takahashi arrives at a cloud i via a wind path, he can choose either to adopt the fluffiness level F_i of the arrived cloud as his "current fluffiness level," or to do nothing and keep his current fluffiness level. This choice can be made each time he arrives at the same cloud. The "current fluffiness level" after this choice is used for the cost calculation when traversing the next wind path.

Find the minimum total time for Takahashi to travel from cloud 1 to cloud N . If cloud N cannot be reached, output -1 .

Constraints

- $2 \leq N \leq 100000$
- $0 \leq M \leq 100000$
- $1 \leq K \leq 5$
- $1 \leq F_i \leq K$ ($1 \leq i \leq N$)
- $1 \leq U_j, V_j \leq N$ ($1 \leq j \leq M$)
- $U_j \neq V_j$ ($1 \leq j \leq M$)
- There may be multiple wind paths connecting the same pair of clouds
- $1 \leq T_j \leq 10^9$ ($1 \leq j \leq M$)
- All input values are integers
- It is guaranteed that the answer is at most 10^{18} (when reachable)

Input

```

 $N$   $M$   $K$ 
 $F_1$   $F_2$   $\dots$   $F_N$ 
 $U_1$   $V_1$   $T_1$ 
 $U_2$   $V_2$   $T_2$ 
 $\vdots$ 
 $U_M$   $V_M$   $T_M$ 

```

- The first line contains the number of clouds N , the number of wind paths M , and the maximum fluffiness level K , separated by spaces.
- The second line contains the fluffiness levels $F_1, F_2, \dots, F_N$ of each cloud, separated by spaces.
- The following M lines give the information of the wind paths. The $(2 + j)$ -th line contains the cloud numbers U_j, V_j connected by the j -th wind path, and the base travel time T_j , separated by spaces.

Output

Output in one line the minimum total time for Takahashi to travel from cloud 1 to cloud N . If cloud N cannot be reached, output -1 .

Sample Input 1

Copy

```

4 4 3
2 1 3 1
1 2 5
2 4 4
1 3 2
3 4 1

```

Sample Output 1

Copy

```

6

```

Sample Input 2

Copy

```

5 3 3
1 2 3 1 2
1 2 10
2 3 5
4 5 7

```

Sample Output 2

[Copy](#)

-1

Sample Input 3

[Copy](#)

```
8 12 5
4 2 5 1 3 2 1 4
1 2 3
1 3 2
2 4 6
3 4 1
4 5 7
5 8 2
4 6 3
6 7 4
7 8 5
2 6 10
3 5 4
1 2 8
```

Sample Output 3

[Copy](#)

19

Sample Input 4

Copy

```
20 30 5
5 4 3 2 1 5 1 2 3 4 5 1 2 3 4 5 1 2 3 1
1 2 100
1 3 7
2 4 6
3 4 2
4 5 10
5 6 3
6 20 50
3 7 8
7 8 1
8 9 1
9 10 1
10 20 20
5 11 4
11 12 4
12 13 4
13 14 4
14 20 4
2 15 30
15 16 2
16 17 2
17 18 2
18 19 2
19 20 2
7 12 9
8 13 9
9 14 9
10 15 9
11 16 9
4 18 25
1 20 1000
```

Sample Output 4

Copy

```
74
```

Sample Input 5

Copy

```
2 1 1
1 1
1 2 1000000000
```

Sample Output 5

[Copy](#)

```
10000000000
```

E - Gift Distribution

Time Limit: 2 sec / Memory Limit: 1024 MiB

Score : 466 pts

Problem Statement

Takahashi is organizing a Christmas party. There are N participants at the party, and Takahashi has decided to give a present to everyone.

Takahashi has prepared M types of presents. There are B_i presents of the i -th type. He must give exactly 1 present to each participant, but he cannot distribute more presents of each type than the number he has prepared. It is guaranteed that the total number of presents ($B_1 + B_2 + \dots + B_M$) is at least N , so it is always possible to give a present to everyone.

Each participant has their own preferences. The j -th participant desires K_j types of presents, and will be satisfied if they receive any one of them. The type numbers of the desired presents are given as $C_{j,1}, C_{j,2}, \dots, C_{j,K_j}$. If $K_j = 0$, that participant will not be satisfied no matter which present they receive. It is allowed to give a participant a present of a type they do not desire, but that participant will not be satisfied.

Takahashi wants to distribute the presents so as to maximize the number of satisfied participants. Find the maximum number of satisfied participants.

Constraints

- $1 \leq N \leq 500$
 - $1 \leq M \leq 500$
 - $1 \leq B_i \leq N$ ($1 \leq i \leq M$)
 - $B_1 + B_2 + \dots + B_M \geq N$
 - $0 \leq K_j \leq M$ ($1 \leq j \leq N$)
 - $1 \leq C_{j,k} \leq M$ ($1 \leq j \leq N, 1 \leq k \leq K_j$)
 - For the j -th participant, $C_{j,1}, C_{j,2}, \dots, C_{j,K_j}$ are all distinct
 - All input values are integers
-

Input

```

 $N$   $M$ 
 $B_1$   $B_2$   $\cdots$   $B_M$ 
 $K_1$   $C_{1,1}$   $C_{1,2}$   $\cdots$   $C_{1,K_1}$ 
 $K_2$   $C_{2,1}$   $C_{2,2}$   $\cdots$   $C_{2,K_2}$ 
 $\vdots$ 
 $K_N$   $C_{N,1}$   $C_{N,2}$   $\cdots$   $C_{N,K_N}$ 

```

- The first line contains N , the number of participants, and M , the number of types of presents, separated by a space.
- The second line contains $B_1, B_2, \dots, B_M$, the number of presents of each type, separated by spaces.
- The following N lines describe each participant's preferences.
 - The $(2 + j)$ -th line contains the number of types K_j that the j -th participant desires, followed by the type numbers $C_{j,1}, C_{j,2}, \dots, C_{j,K_j}$, separated by spaces. If $K_j = 0$, no type numbers are given, and only K_j is written.

Output

Output the maximum number of satisfied participants in a single line.

Sample Input 1

[Copy](#)

```

3 2
2 2
1 1
1 1
1 2

```

Sample Output 1

[Copy](#)

```

3

```

Sample Input 2

[Copy](#)

```

4 2
1 3
1 1
1 1
1 1
1 1
1 1

```

Sample Output 2

[Copy](#)

```
1
```

Sample Input 3

[Copy](#)

```
6 3
2 2 3
1 1
1 1
2 1 2
1 2
1 3
0
```

Sample Output 3

[Copy](#)

```
5
```

Sample Input 4

[Copy](#)

```
10 5
2 2 2 2 3
2 1 2
2 1 2
1 3
1 3
2 3 4
2 4 5
1 5
1 5
2 1 5
0
```

Sample Output 4

[Copy](#)

```
9
```


Sample Input 5

[Copy](#)

```
1 1
1
0
```

Sample Output 5

[Copy](#)

```
0
```